Amrita School of Computing

Amritapuri Campus

Department of Computer Science and Engineering

B.Tech Computer Science and Engineering

23CSE498 – Project Phase II

Design and Solution Submission

Project Title Predictive Reaction Simulation for Adaptive, Multi-Turn, Multilingual Counterspeech Generation

Project Guide Ms. Siji Rani S

Team Members

Team No.	Roll Number	Name of Student
8	AM.SC.U4CSE23304	Advyth Vaman Akalankam
8	AM.SC.U4CSE23331	Madana Lakshmi Tanmayi
8	AM.SC.U4CSE23344	Rayudu Vamsi Krishna
8	AM.SC.U4CSE23351	Sangamreddi Hemarshitha

Academic Year: 2026–2027

Amrita Vishwa Vidyapeetham

1 Sustainable Development Goal (SDG) Mapping

Primary SDG: SDG 16 – Peace, Justice and Strong Institutions

Secondary SDG(s) (Optional): SDG 10 – Reduced Inequalities; SDG 5 – Gender Equality

Justification (100–150 words):

Online hate speech directly undermines SDG 16's goal of building peaceful, inclusive societies by normalizing hostility toward targeted groups and eroding safe civic discourse. This project contributes to that goal by developing an AI-assisted counterspeech system that helps de-escalate hostile online exchanges rather than simply removing content, supporting healthier digital public spaces. The project also supports SDG 10 (Reduced Inequalities), since hate speech disproportionately targets marginalized communities based on religion, ethnicity, and gender, and effective counterspeech tools can reduce the disproportionate harm these groups face online. SDG 5 (Gender Equality) is a relevant secondary goal because a significant share of hate speech in the underlying datasets (CONAN, Gab, Reddit) targets women and gender minorities, and extending counterspeech generation to multiple Indian languages further ensures these protections are not limited to English-speaking, Western online communities.

SDG	Goal
1	No Poverty
2	Zero Hunger
3	Good Health and Well-being
4	Quality Education
5	Gender Equality
6	Clean Water and Sanitation
7	Affordable and Clean Energy
8	Decent Work and Economic Growth
9	Industry, Innovation and Infrastructure
10	Reduced Inequalities
11	Sustainable Cities and Communities
12	Responsible Consumption and Production
13	Climate Action
14	Life Below Water
15	Life on Land
16	Peace, Justice and Strong Institutions
17	Partnerships for the Goals

2 Project Milestones and Timeline

Semester	Major Activity / Milestone	Expected Deliverable	Expected Completion	Status
S6	Project domain exploration	Problem domain identified	15 Feb 2026	Completed
S6	Literature review	Initial literature survey	15 Mar 2026	Completed
S6	Baseline implementation	Reference implementation completed	30 Apr 2026	Completed
S7 (Submission 1)	Literature survey	Comprehensive literature survey	15 Jul 2026	Completed
S7 (Submission 1)	Research gap identification	Research gap statement	20 Jul 2026	Completed
S7 (Submission 1)	Problem statement	Finalized objectives	25 Jul 2026	Completed
S7 (Submission 2)	System design	Architecture and workflow	09 Aug 2026	Completed
S7	Prototype implementation	Functional prototype	15 Oct 2026	Planned
S7	Module integration	Integrated system	15 Nov 2026	Planned
S7	Testing and validation	Experimental results	15 Dec 2026	Planned
S8	System enhancement	Improved system	31 Jan 2027	Planned
S8	Performance evaluation	Performance analysis	28 Feb 2027	Planned
S8	Research publication	Paper submission	20 Mar 2027	Planned
S8	Dissertation writing	Final report	10 Apr 2027	Planned
S8	Final demonstration	Presentation and viva	20 Apr 2027	Planned

Table 1: Overall Project Timeline

3 System Architecture

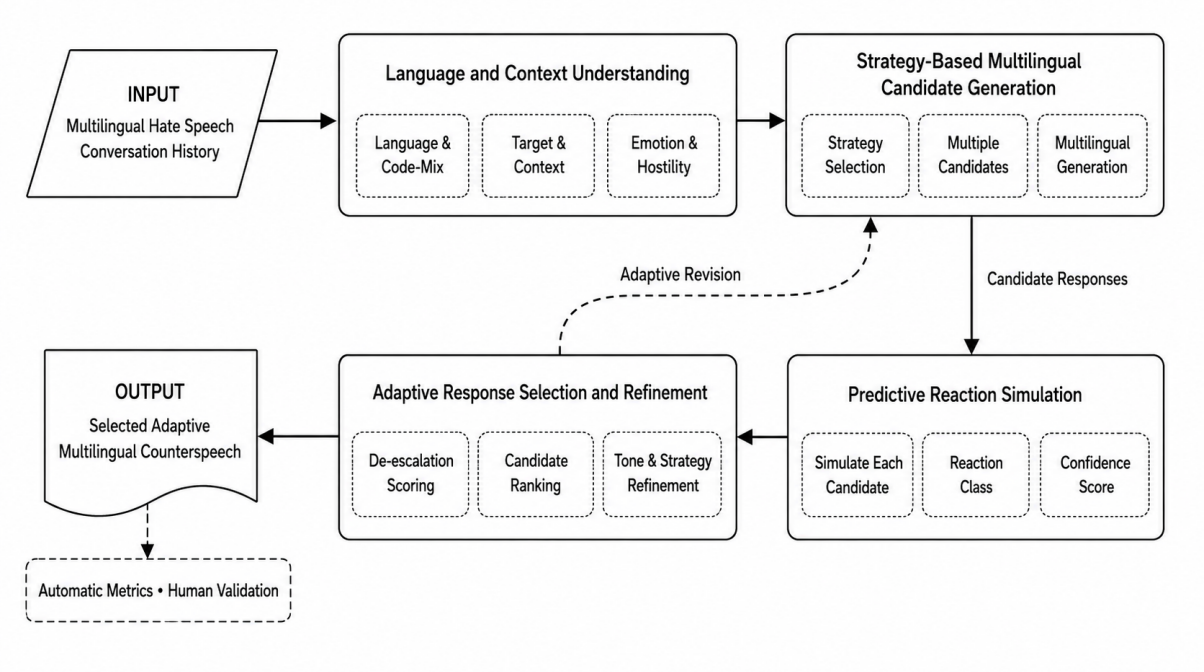


Figure 1: Architecture of the proposed adaptive counterspeech generation system.

Architecture Description

The architecture follows a closed-loop pipeline of four core processing modules between an input and an output stage, traceable directly to the project objectives stated in the Phase 1 submission: a hate-speech comment (with any conversation history) first passes through Language and Context Understanding, whose output conditions Strategy-Based Multilingual Candidate Generation, which produces several candidate replies for Predictive Reaction Simulation to evaluate; the resulting predictions feed Adaptive Response Selection and Refinement, which either finalizes the best candidate as the system's output or triggers an Adaptive Revision loop back to candidate generation. The finalized reply is also logged for automatic and human evaluation. Each module is described in detail in Section 4, and interfaces between modules are text-based (JSON payloads), keeping every module independently testable and replaceable.

4 Module Description

Module	Description
Module 1: Language and Context Understanding	Analyzes the incoming hate-speech comment (and conversation history, if any) along three dimensions: detects the language or code-mixed language combination (*Language & Code-Mix*); identifies the target group and surrounding conversational context (*Target & Context*); and characterizes the emotional tone and hostility level of the input (*Emotion & Hostility*). These outputs condition every downstream module.
Module 2: Strategy-Based Multilingual Candidate Generation	Selects an appropriate counterspeech strategy based on the understanding stage's output (*Strategy Selection*), generates several distinct candidate replies for that strategy using a large language model (*Multiple Candidates*), and produces each candidate in the language identified upstream (*Multilingual Generation*). Also receives the *Adaptive Revision* feedback signal from Module 4 and regenerates candidates when triggered.
Module 3: Predictive Reaction Simulation	For each candidate reply generated by Module 2, role-plays the original hate-speaker persona to produce a simulated response (*Simulate Each Candidate*), classifies that simulated response into a reaction category such as escalating, neutral, or constructive (*Reaction Class*), and computes a *Confidence Score* indicating how reliable that prediction is.
Module 4: Adaptive Response Selection and Refinement	Consumes the simulated reactions and confidence scores for all candidates and scores each by its predicted de-escalation potential (*De-escalation Scoring*), ranks the candidates and selects the strongest one (*Candidate Ranking*), and adjusts tone or strategy where needed (*Tone & Strategy Refinement*). If no candidate is acceptable, this module triggers the *Adaptive Revision* feedback loop back to Module 2 rather than finalizing a weak reply.
Module 5: Evaluation (Automatic Metrics & Human Validation)	Runs alongside the core pipeline on the finalized output: scores the selected reply using automatic classifiers (toxicity, counterspeech quality, argument strength, adapted from the base paper) and logs a random sample of outputs for the planned human evaluation study comparing adaptive, strategy-based replies against plain zero-shot replies.

5 System Workflow

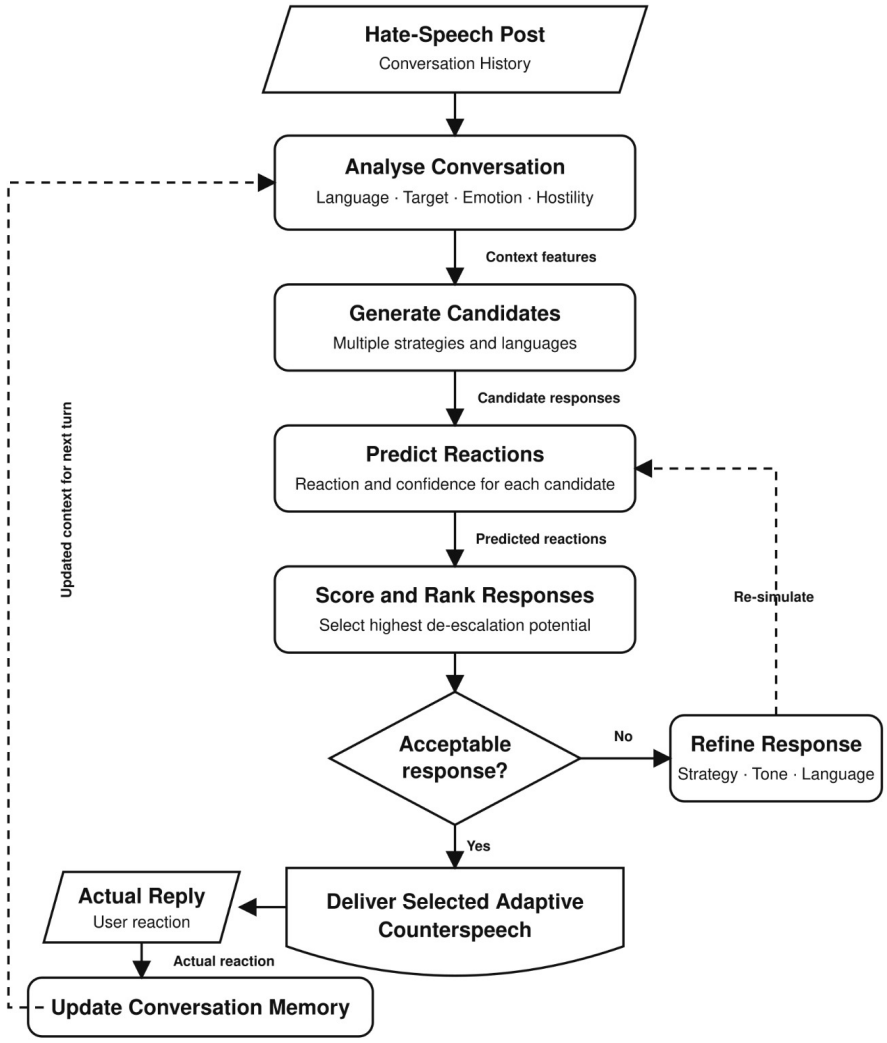


Figure 2: End-to-end operational workflow of counterspeech generation pipeline.

Proposed System Description

The proposed system is a prediction-based, adaptive, multi-turn, and multilingual framework for generating effective counterspeech against online hate speech. It receives the current hate-speech post, previous conversation turns, and the author's earlier reactions. After preprocessing, the system detects the language and code-mixed content and analyses the target, context, emotion, intent, and hostility level. Based on this analysis and conversation memory, it generates multiple multilingual counterspeech candidates using strategies such as empathy, questioning, positive messaging, warning, alternative perspectives, and fact-based correction. A reaction-simulation component predicts whether each candidate may lead to acceptance, rejection, disengagement, or escalation. The can-

didates are evaluated for de-escalation potential, relevance, politeness, toxicity, linguistic quality, and cultural appropriateness. The best response is selected, while unsuitable responses are adaptively refined. After delivery, the actual reaction is compared with the prediction, and the conversation memory is updated to improve responses in subsequent turns.

6 Algorithms / Methodology

Proposed Algorithm / Methodology Description

The core methodological contribution is a **multi-candidate Generate → Simulate → Select-and-Refine** pipeline that replaces single-shot counterspeech generation with a process that generates several strategy-conditioned candidates, predicts a social outcome for each, and adaptively selects or revises among them. For a given hate-speech comment (and, in multi-turn settings, its conversation history), the system first extracts language, target/context, and emotion/hostility signals. A counterspeech strategy is then selected and used to generate multiple candidate replies in the detected language. Each candidate is independently passed through a reaction simulator that predicts how the original hate-speaker would most plausibly respond, together with a reaction class and confidence score. All candidates are then scored for predicted de-escalation potential and ranked; the top-ranked candidate is finalized if it clears an acceptability threshold. If no candidate is acceptable, an Adaptive Revision signal triggers regeneration of candidates under a revised strategy, and the simulate-score-rank process repeats, up to a fixed maximum number of revision rounds to bound cost and latency. This directly operationalizes Objectives 1 and 2 from the Phase 1 submission. The same pipeline is reused unchanged for multi-turn conversations (Objective 3) by conditioning candidate generation on conversation memory, and for multilingual generation (Objective 4) by conditioning the Understanding and Candidate Generation modules on the detected language.

Pseudocode / Mathematical Formulation

```
function GENERATE_COUNTERSPEECH(comment, history, max_rev=3, k=4)
  lang, target, emotion ← UnderstandInput(comment, history)
  strategy ← SelectStrategy(target, emotion)
  for r = 1 to max_rev do
    candidates ← GenerateCandidates(comment, history, lang, strategy, k)
    for each c in candidates do
      reaction[c] ← ReactionSimulator(comment, c, lang)
```

```
      score[c] ← DeescalationScore(reaction[c])
    end for
    best ← argmax(score)
    if score[best] ≥ threshold then
      return best, score[best], r
    end if
    strategy ← ReviseStrategy(strategy, reaction[best])
  end for
  return best, score[best], max_rev
end function
```

Here, UnderstandInput, GenerateCandidates, and ReactionSimulator are LLM calls parameterized by prompt templates conditioned on `lang`; DeescalationScore is a fine-tuned scoring function over the reaction class and confidence score. The outer loop terminates either when an acceptable candidate is found or when `max_rev` revision rounds are exhausted, whichever comes first. Since k candidates are simulated per round, the worst-case cost per reply is $O(\text{max_rev} \times k)$ Reaction Simulator calls, bounding latency for a fixed k and `max_rev`.

7 Technology Stack

Category	Version	Technology / Tool
Programming Language	Python 3.11	Core pipeline, model orchestration, evaluation scripts
Framework	FastAPI	Backend REST API serving the generate-reply endpoint
Frontend	React 18 / Next.js	Demo web interface for entering comments and viewing selected replies
Backend	FastAPI + Uvicorn	Orchestrates Understanding, Candidate Generation, Reaction Simulation, and Selection/Refinement calls
Database	PostgreSQL 16	Stores conversations, turns, candidates, and evaluation logs
Libraries	LangChain / raw API SDKs, Sentence-Transformers, HuggingFace Transformers	LLM orchestration, embeddings, fine-tuning the de-escalation scorer
IDE / Development Environment	VS Code	Primary development environment
Version Control	Git / GitHub	Source control and collaboration
Cloud / Deployment Platform	Docker + a cloud VM (e.g. AWS/GCP free tier)	Containerized deployment of the prototype service
Operating System	Ubuntu 22.04 LTS (dev & deployment)	Development and server environment
Other Tools	Postman, Turnitin (for report checks)	API testing; academic integrity verification

8 Hardware and Software Requirements

Hardware Requirements

Component	Specification
Processor	Quad-core CPU (Intel i5 / AMD Ryzen 5 or equivalent) or higher
RAM	16 GB minimum (32 GB recommended for local model experimentation)
Storage	50 GB free disk space (datasets, logs, model checkpoints)
GPU (If Applicable)	Not required for API-based LLM calls; an NVIDIA GPU with $\geq$ 12 GB VRAM is recommended only if local fine-tuning of the de-escalation scorer is performed
Internet Connectivity	Stable broadband connection (required for LLM API calls)
Other Hardware	None

Software Requirements

Software	Version / Details
Operating System	Ubuntu 22.04 LTS / Windows 11 / macOS (development); Ubuntu 22.04 LTS (deployment)
Programming Language(s)	Python 3.11, JavaScript/TypeScript (frontend)
Framework(s)	FastAPI, React / Next.js
Database	PostgreSQL 16
IDE / Editor	Visual Studio Code
Libraries / Packages	HuggingFace Transformers, Sentence-Transformers, LangChain, Pandas, NumPy, scikit-learn
Other Dependencies	Docker, Git, LLM provider API access (e.g. Llama-3/Mistral via an inference API)

9 Design Considerations (Non-Functional)

Design Aspect	Description
Scalability	The stateless Understanding, Candidate Generation, and Reaction Simulation calls can be horizontally scaled behind the API layer, and candidate simulation for a single turn can be parallelized across the k candidates; conversation state is isolated per `conversation_id` in the database, allowing multiple conversations to be processed concurrently.
Performance	The Adaptive Revision loop is capped at a fixed number of rounds, and each round simulates a fixed number of candidates in parallel, bounding worst-case latency to $O(\text{max_rev} \times k)$ LLM calls; asynchronous request handling in FastAPI allows concurrent processing of multiple conversations.
Security	LLM API keys are stored as environment variables and never committed to source control; user-submitted text is sanitized before being inserted into prompts to reduce prompt-injection risk against the Understanding, Candidate Generation, and Reaction Simulation modules.
Reliability	If an LLM API call fails or times out for one candidate, the pipeline proceeds with the remaining successfully generated candidates rather than failing the request outright, and logs the failure for review.
Maintainability	Each module (Language and Context Understanding, Candidate Generator, Reaction Simulator, Response Selector/Refiner) is implemented as an independently testable service behind a defined JSON interface, so individual components (e.g. swapping the LLM provider or de-escalation scorer) can be replaced without changing the rest of the pipeline.
Privacy	Hate-speech comments and generated replies are logged only with anonymized identifiers. The prototype will not collect personally identifiable information from users or real social-media accounts; public research datasets (CONAN, Reddit, Gab) will be used only for research and evaluation, in accordance with their respective terms of use.
Ethical Considerations (If Applicable)	Consistent with the ethics guidance in the base paper (Saha et al., 2024), this project does not aim to deploy a fully automatic, unsupervised counterspeech pipeline; the system is positioned as a decision-support / human-in-the-loop tool, and the human evaluation study is designed to surface cases

10 Expected Outcome

By the end of the two-semester project, the following outcomes are expected:

- A working software prototype implementing the strategy-based, multi-candidate generate → simulate → select-and-refine counterspeech pipeline, extended to handle multi-turn conversations and at least two languages (English and one Indian language), accessible through a web-based demo interface.
- Quantitative evidence, from both automatic metrics and a human evaluation study (approx. 100–150 rated samples), on whether adaptive, reaction-simulation-driven candidate selection produces counterspeech that is more likely to de-escalate conflict than single-shot zero-shot generation.
- A documented, human-validated synthetic multi-turn hate-speech/counterspeech conversation dataset, suitable for reuse in future research on conversational counterspeech.
- A written dissertation and, subject to result quality, a paper draft suitable for submission to one of the identified venues (ICON, EMNLP, or COLING), extending the base paper's zero-shot counterspeech generation study with strategy-based multilingual candidate generation, predictive reaction simulation, and adaptive selection and refinement.

11 References

1. P. Saha, A. Agrawal, A. Jana, C. Biemann, and A. Mukherjee, "On Zero-Shot Counterspeech Generation by LLMs," in *Proc. LREC-COLING 2024*, pp. 12443–12454, 2024.

2. P. Saha, K. Singh, A. Kumar, B. Mathew, and A. Mukherjee, "CounterGeDi: A Controllable Approach to Generate Polite, Detoxified and Emotional Counterspeech," in *Proc. IJCAI 2022*, pp. 5157–5163, 2022.

3. A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

4. J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proc. 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

5. A. Singh and R. Thakur, "Generalizable Multilingual Hate Speech Detection on Low Resource Indian Languages using Fair Selection in Federated Learning," in *Proc. NAACL 2024*, pp. 7211–7221, 2024.

6. Y.-L. Chung, E. Kuzmenko, S. S. Tekiroğlu, and M. Guerini, "CONAN — COunter NArratives through Nichesourcing: a Multilingual Dataset of Responses to Fight Online Hate Speech," in *Proc. ACL 2019*, pp. 2819–2829, 2019.

7. J. Qian, A. Bethke, Y. Liu, E. Belding, and W. Y. Wang, "A Benchmark Dataset for Learning to Intervene in Online Hate Speech," in *Proc. EMNLP-IJCNLP 2019*, pp. 4755–4764, 2019.

Faculty Remarks

Guide Remarks

Project Guide

Siji Rani S

Name: _____

Signature: _____

Project Coordinator

Name: _____

Signature: _____